

LC 1722: Minimize Hamming Distance After Swap Operations

Google Interview Problem - Notes & Implementation

Problem Example

```
s = [1, 2, 3, 4]
d = [2, 1, 4, 5]
allowed swaps = [(0, 1), (2, 3)]
```

Then:

```
s can become [2, 1, 4, 3]
comparing to [2, 1, 4, 5]
hamming = 1 (minimum)
```

Key Observations

(S1) union the indices

(S2) now inside one component the order does not matter , only the frequency matters because any 2 position within a component are swappable

Algorithm Flow

[union indices]



[leader]



[collect indices of comp]



[count source frequencies]



[try matching target]



if target value exists

frequency--

**else
ans++**

Implementation (C++)

```
#include<bits/stdc++.h>
using namespace std;

class dsu{
public:
    vector<int> parent;
    vector<int> size;
    dsu(int n){
        parent.resize(n);
        size.resize(n,1);

        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findparent(int node){
        if(parent[node]==node){
            return node;
        }

        parent[node]=findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u,int v){
        int pu=findparent(u);
        int pv=findparent(v);

        if(pu==pv) return;

        if(size[pu]<size[pv]) swap(pu,pv);

        parent[pv]=pu;
        size[pu]+=size[pv];
    }
};
```

```

    }
};

class Solution {
public:
    int minimumHammingDistance(vector<int>& source, vector<int>& target, vector<vector<int>>& s) {
        int n=source.size();
        dsu ds(n);

        for(auto &it:swaps){
            ds.unionit(it[0],it[1]);
        }

        unordered_map<int,vector<int>> mp;

        for(int i=0;i<n;i++){

            int boss=ds.findparent(i);
            mp[boss].push_back(i);
        }

        int ans=0;
        for(auto &it:mp){

            vector<int> indices=it.second;
            unordered_map<int,int> freq;
            //ab ek component me koi bhi value kisi indices pe bhi place ho skta
            //hence source waala bas target waale me hona chahiye

            //source ki frequency counting kr lenge uss component me
            //phir target me se ek ek krke match krte jaayenge

            for(auto &it:indices){
                freq[source[it]]++;
            }

            for(auto &it:indices){
                if(freq[target[it]]>0) freq[target[it]]--;
                else ans++;
            }

        }

        return ans;
    }
};

```